

SmartLogix – Plataforma Inteligente para la Gestión Logística de eCommerce.

Diseño de Arquitectura y Patrones de Microservicios.

Sección: 003D.

Docente: Claudio Esteban Rojas Rodriguez.

Integrantes:

- **Gabriel Jeria.**
- **Vicente Palacios.**
- **Isak Chacana.**

Introducción.

Contexto.

El auge del comercio electrónico ha impulsado la necesidad de sistemas más eficientes y escalables para la gestión de inventarios, pedidos y envíos. SmartLogix es una startup en crecimiento que busca ofrecer una plataforma moderna e integrada para la optimización de procesos logísticos en pequeñas y medianas empresas (PYMEs) de eCommerce. La empresa ha identificado que muchas de estas PYMEs aún dependen de sistemas monolíticos y procesos manuales, lo que genera inconsistencias en la información, retrasos en la entrega y dificultades para manejar picos de demanda.

Para resolver estos problemas, SmartLogix quiere desarrollar una solución basada en microservicios, con una capa frontend moderna y flexible, y un backend escalable y seguro, permitiendo a las PYMEs gestionar su operación logística con mayor eficiencia y reducir costos operativos.

Actualmente, las empresas que utilizan soluciones logísticas tradicionales enfrentan desafíos como sincronización deficiente de inventarios, errores en la asignación de pedidos, y demoras en la coordinación de envíos. Muchos de estos problemas surgen porque sus sistemas monolíticos no permiten la automatización ni integración con nuevas tecnologías. SmartLogix ha decidido desarrollar una arquitectura moderna basada en microservicios para garantizar la escalabilidad y flexibilidad de su plataforma, facilitando la automatización de la gestión logística.

Desarrollo

Arquitectura

El sistema SmartLogix está compuesto por un componente Frontend desarrollado en React + Vite, un Backend For Frontend (BFF) encargado de centralizar las peticiones del cliente y de múltiples microservicios y 4 servicios hechos en Java con framework de Spring Boot. Cada microservicio posee su propia base de datos, continuando con el principio de desacoplamiento y autonomía de datos.

Todo este sistema está implementado con Kubernetes, el cual ajusta el escalado de cada servicio dependiendo de su carga

Además, la comunicación entre los componentes se realiza mediante APIs REST, permitiendo una integración flexible y facilitando futuras ampliaciones del sistema.

Se usa Flyway para el manejo de esquemas dentro de cada base de datos de cada microservicio, asegurando la consistencia y robustez de la base de datos y el apropiado control de versionado dentro de esta.

También los microservicios tienen una implementación de repository pattern, con Java Persistence API (JPA) el cual interactúa con su base de datos PostgreSQL correspondiente, el cual esta alojado en un Persistent Volume Claim (PVC) dentro de Kubernetes.

Las pruebas unitarias son realizadas a través de junit, los cuales verifican la integridad del código a la hora de desplegarlo.

API Gateway

El diseño considera el uso de un API Gateway para centralizar el acceso a los servicios del sistema. Este componente permite gestionar rutas, autenticación, seguridad y monitoreo de solicitudes realizadas hacia los distintos microservicios. Para este proyecto usamos KrakenD como nuestro gateway, asegurando que cualquier conexión no segura sea permitida, y aislando el backend del resto de internet.

Frontend

El Frontend está desarrollado con Typescript con el ambiente Node JS, usando las librerías de React y empaquetada con Vite, este sigue un modelo de arquitectura en capas, separando sus obligaciones en Api (capa de servicio), ViewModels (capa lógica) y pages (capa vista o visual). Esto permite separar la lógica de presentación de la interfaz gráfica, facilitando la reutilización y mantenimiento de componentes.

El Frontend tiene un pantalla de ingreso y autentifica la identidad del usuario, tras eso procede tener una pantalla de inventario, órdenes y pedidos, los cuales visualizan y crean pedidos de API, interactuando con el Backend a través de la API.

Backend for Frontend (BFF)

El BFF actúa como una capa intermedia entre el Frontend y los microservicios. Su función principal es adaptar las respuestas de los servicios a las necesidades específicas de la interfaz de usuario, reduciendo la complejidad del cliente y centralizando ciertas operaciones.

Para nuestra aplicación, el Back for Frontend se encarga de verificar la identidad del usuario a través del microservicio de usuarios, acompañado con una implementación de JWT, asegurándose de que el que acceda a los servicios ofrecidos por SmartLogix tenga autorización para accederlos.

El BFF está separado en varias capas usando como base el patrón de arquitectura de Modelo Servicio Controlador, implementando también la capa Cliente para

interactuar con el resto de microservicios, DTOs para el transporte de datos entre el frontend y el mismo BFF, y una capa de seguridad cual contiene la implementación de tokens JWT.

Como el BFF es la única parte pública de nuestros servicios, esta le acompaña una documentación swagger completa con todo lo que un cliente necesitaría para interactuar con ella a nivel API.

Users

El microservicio ms-user se encarga de almacenar y gestionar los usuarios dentro del sistema, además de en conjunto con el BFF, ayuda a la autenticación de estos mismos.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/users, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (user-db), siguiendo el principio de autonomía de datos por microservicio.

Inventory

Este se encarga de manejar los productos dentro del inventario, manteniendo el stock, avisando cuando haya stock bajo y el precio de cada producto.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/inventory, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (inventory-db), siguiendo el principio de autonomía de datos por microservicio.

Orders

Este se encarga del manejo de órdenes realizadas por los usuarios de un marketplace, incluyendo estado, dirección, a quien se dirige y el total de la orden, además de enviar actualizaciones de inventario cuando se haya realizado un pedido.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/orders, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (orders-db), siguiendo el principio de autonomía de datos por microservicio.

Restock

El microservicio ms-restock gestiona las solicitudes de reposición de stock de la plataforma SmartLogix. Permite crear, consultar, aprobar, rechazar y eliminar solicitudes asociadas a un ítem y bodega, manteniendo un ciclo de vida controlado mediante el atributo estado; PENDIENTE, APROBADA, RECHAZADA y COMPLETADA.

Sigue el arquetipo MSC, con persistencia en una base de datos PostgreSQL propia, gestionada mediante Flyway, respetando el principio de autonomía de datos por microservicio. La API REST se expone en el puerto 9093 bajo /api/restock.

Se integra con ms-inventory mediante RestTemplate: al crear una solicitud válida que el ítem existe en el inventario, y al probarla actualiza automáticamente el stock sumando la cantidad solicitada, dándole coherencia real al flujo de negocio.

El BFF consume este servicio a través de un cliente Feign (RestockServiceClient) y lo expone bajo /api/bff/restock hacia frontend y el API Gateway KrakenD. Las pruebas unitarias se implementan con JUnit y Mockito sobre la capa de servicio, cubriendo los casos principales del ciclo de vida de una solicitud con una cobertura superior al 60%.